🎓 جزوه طلایی سیستم‌عامل

فصل ۱۰: حافظه مجازی (Virtual Memory) - بخش اول

۱. مقدمه و تعریف پایه

در فصل‌های قبل (مدیریت حافظه)، استراتژی‌ها یک محدودیت بزرگ داشتند: **«کل پردازه (Process) باید قبل از اجرا، داخل حافظه قرار می‌گرفت.»** اما **حافظه مجازی (Virtual Memory)** این محدودیت را می‌شکند.

🔥 **تعریف حافظه مجازی:** تکنیکی که اجازه می‌دهد پردازه‌ها اجرا شوند، حتی اگر **به طور کامل** در حافظه نباشند.

✅ **مزایای اصلی این تکنیک (چرا شاهکار است؟):**

1. **حذف محدودیت اندازه:** برنامه می‌تواند **بزرگتر** از حافظه فیزیکی (RAM) موجود باشد.
2. **انتزاع (Abstraction):** حافظه منطقی (دید برنامه‌نویس) از حافظه فیزیکی جدا می‌شود. برنامه‌نویس یک آرایه حافظه بی‌نهایت بزرگ می‌بیند و نگران پر شدن RAM نیست.
3. **اشتراک‌گذاری آسان:** راحت‌تر می‌توان فایل‌ها و کتابخانه‌ها را بین برنامه‌ها به اشتراک گذاشت.
4. **ایجاد پردازه کارآمد:** ساختن Process جدید سریع‌تر و بهینه‌تر می‌شود.

۲. چرا اجرای برنامه "بدون بارگذاری کامل" مفید است؟

شاید بپرسید چرا کل برنامه را بارگذاری نکنیم؟ تحقیقات نشان داده در بسیاری از موارد، کل کد برنامه همزمان نیاز نیست:

- **کدهای خطا:** کدهایی که فقط در شرایط نادر اجرا می‌شوند.
- **آرایه‌های بزرگ:** مثلاً آرایه ۱۰۰×۱۰۰ تعریف شده اما فقط ۱۰×۱۰ آن استفاده می‌شود.
- **امکانات خاص:** آپشن‌هایی که کاربر به ندرت انتخاب می‌کند.

💎 **منافع اجرا با بارگذاری جزئی (Partial Loading):**

1. **برنامه‌های بزرگتر:** دیگر محدود به حجم RAM نیستیم (فضای آدرس مجازی بسیار بزرگ).

2. **افزایش درجه چندبرنامگی (Degree of Multiprogramming):** چون هر برنامه فضای کمتری اشغال می‌کند، برنامه‌های بیشتری همزمان اجرا می‌شوند ⬅ **افزایش بهره‌وری CPU.**
3. **سرعت بیشتر (I/O کمتر):** چون حجم کمتری از برنامه باید از دیسک بارگذاری (Load) یا مبادله (Swap) شود، سرعت اجرا بالا می‌رود.

۳. ساختار حافظه مجازی

حافظه مجازی دیدگاه **منطقی** و **فیزیکی** را جدا می‌کند:

دیدگاه فیزیکی (سخت‌افزار)	دیدگاه منطقی (برنامه‌نویس)
برنامه در "قاب صفحه‌های" (Page Frames) پراکنده در حافظه پخش شده است.	برنامه به صورت پیوسته از آدرس ۰ تا Max قرار دارد.

نکته مهم: وظیفه تبدیل (Map) آدرس‌های منطقی به فیزیکی بر عهده واحد **MMU** (Memory Management Unit) است.

✸ ۴. فضای آدرس خلوت (Sparse Address Space)

در ساختار مجازی، فضای بین پشته (Stack) و هیپ (Heap) می‌تواند خالی باشد:

- **Heap:** از پایین به بالا رشد می‌کند (تخصیص پویا).
- **Stack:** از بالا به پایین رشد می‌کند (فراخوانی توابع).
- **Hole (حفره):** فضای خالی بین این دو.

✓ **مزیت فضای خلوت:** این حفره‌ها (Holes) بخشی از آدرس مجازی هستند اما تا زمانی که Heap یا Stack رشد نکنند، **هیچ حافظه فیزیکی اشغال نمی‌کنند.**

- کاربرد: پر کردن این فضاها با کتابخانه‌های پویا (Dynamic Libraries) در حین اجرا.

۵. اشتراک‌گذاری حافظه (Memory Sharing)

حافظه مجازی اشتراک‌گذاری منابع بین پردازه‌ها را بسیار ساده می‌کند (مثلاً اشتراک‌گذاری کتابخانه استاندارد C):

الف) اشتراک کتابخانه‌های سیستمی (System Libraries)

- **چطور کار می‌کند؟** چندین پردازه تصور می‌کنند که کتابخانه را در فضای اختصاصی خود دارند.

- **واقعیت:** تمام آن‌ها به یک نسخه فیزیکی واحد در RAM اشاره می‌کنند (معمولاً به صورت **فقط-خواندنی**).

ب) حافظه مشترک (Shared Memory)

- دو پردازه می‌توانند یک ناحیه مشترک برای تبادل اطلاعات بسازند.
- هر پردازه این ناحیه را بخشی از آدرس مجازی خودش می‌بیند، اما هر دو به یک فریم فیزیکی مشترک دسترسی دارند (قابلیت خواندن/نوشتن).

ج) تسریع ایجاد پردازه (Process Creation)

با استفاده از تکنیک‌هایی (مثل Copy-on-Write که بعدا می‌خوانیم) و فراخوانی سیستمی fork()، می‌توان صفحات را در ابتدا به اشتراک گذاشت و کپی نکرد تا سرعت ایجاد پروسه جدید بسیار بالا برود.

📝 خلاصه برای شب امتحان (جمع‌بندی)

- **حافظه مجازی:** جداسازی حافظه منطقی از فیزیکی + امکان اجرای برنامه بدون بارگذاری کامل.
- **مزیت کلیدی:** برنامه می‌تواند بزرگتر از RAM باشد + CPU بیشتر کار می‌کند (چندبرنامگی بیشتر).
- **MMU:** مسئول تبدیل آدرس مجازی به فیزیکی.
- **فضای خلوت (Sparse):** فضای خالی بین پشته و هیپ که فیزیکی را اشغال نمی‌کند مگر نیاز شود.
- **اشتراک‌گذاری:** یک نسخه فیزیکی از کتابخانه، چندین نگاشت مجازی در پردازه‌های مختلف.